

Groovy-101

Getting Started with Groovy

Agnibha Bose | feedback@digit.in

T

he following tutorial is for those who have a little hands on experience with some programming

language (preferably Java). Essentially, Groovy provides a great expansion over Java as a programming language. It provides some of the flexibility of the loosely typed languages like scripting languages to Java.

REQUIREMENTS

We are using IntelliJ as the IDE. You can use anything other than IntelliJ too. But IntelliJ comes with a predefined Groovy Support as a framework. We will also be providing Linux Support in this tutorial, though you can find Groovy support for your Operating System in their downloads page.

INSTALLATION

Installing Groovy is very easy on Linux. If you have Sdkman in your system then it will be a cakewalk otherwise we would suggest installing SDKMan first and then installing Groovy from that. If you are into using Java and Java Oriented Languages then SDKMan is a good utility to work with. To install SDKMan, go to your terminal and add the following script.

```
$ curl -s get.sdkman.io | bash
```

This will install SDKMan. After finishing with the installation process add the following code into your `~/.runcom` file (.bashrc, .zshrc)`

```
$ export HOME="$HOME/.sdkman/  
bin/sdkman-init.sh"
```

Or you can simply open a new terminal and execute the following code.

```
$ source "$HOME/.sdkman/bin/  
sdkman-init.sh"
```

After you are finished with SDKMan then just type in the following script to install Groovy .

```
$ sdk install groovy
```

After the installation you can test the installed version by typing the following into the terminal.

```
$ groovy -version
```

And that's all there is to do.

MATH AND BASICS

Now to demonstrate basic and mathematical operations in Groovy we are going to create a new file. The extension is `.groovy`. For the filename, let's go with `MathsAndBasics.groovy`.

Within the file we are going to create a new Class `MathsAndBasics`. People from Java know why the filename and the public class name should be the same. For others, here's a brief explanation – the code that we write will first go through a compilation step where a byte code (in this case a `.class` file) will get generated. That `.class` file will be executed eventually.

Now within the class we will create a main method, which will look like `static void main(String[] args){}`

Now over here `static` means this is a method or a function that belongs to the class. `void` means the method will not return anything. `main` is the name of the method which takes a String array as an argument from the command line. The main method is executed whenever a class is executed.

To print something, type

```
println("Hello World");
```

To define a variable you need to write `def` followed by the name of the variable. Groovy is a dynamically typed language. So unlike Java the type

of the variable depends on the runtime, because of that the same variable that you have used to store a string can also be used to hold a number, as shown at <https://digit.in/Groovy0>. To run the code just type `$ groovy <FilePath>` and the output will be:

```
Hello World  
Name as a String -> Agnibha  
name as a Number -> 40
```

Now the basic mathematical operations will work like Java or any other language. But every number will have some methods associated with it. In Groovy, numbers are treated as objects. This is easy to compare with JavaScript because in JavaScript everything is an object, similarly in Groovy things are treated as object. Numbers are the instances of the Integer class. As numbers are objects, they inherit some predefined methods from the Groovy Integer Definition. Some examples of such methods are in this code sample: <https://digit.in/Groovy1>.

From the above piece of code you can see that numbers inherit some typical methods like `intdiv`, `plus`, `minus`, `multiply` etc. Instead of these methods you can also use the operators. For integer division you have to use `intdiv`. Groovy by default provides division in floating points. Output of the above code is below.

```
5 + 6 => 11  
5 - 6 => -1  
5 * 6 => 30  
5 / 6 => 0  
5 % 6 => 5  
5.1 + 6.1 => 11.2  
5.1 - 6.1 => -1.0  
5.1 * 6.1 => 31.11  
5.1 / 6.1 => 0.8360655738
```

The order of mathematical operations in Groovy is the same as standard mathematics – brackets, followed by division, multiplication, addition